

Bản đồ codebase DataForge

Tài liệu này dẫn bạn đi qua từng thư mục, từng file: nó dùng để làm gì, và nó nối với phần còn lại ra sao.

LƯU Ý VỀ TRẠNG THÁI REPO

Đây đang là **bộ khung**. Phần *khai báo* (model dữ liệu, schema, cấu hình) và phần *nối dây* (tạo app, dựng graph, Compose) là code chạy được thật. Phần *nghiệp vụ* hầu hết mới là chỗ trống, đánh dấu bằng `TODO` + `raise NotImplementedError`. Điều này **có chủ đích**: chữ ký hàm và docstring chính là bản hợp đồng mô tả việc cần làm. Trong tài liệu này tôi ghi rõ *thật* hay *chỗ trống* cho từng file.

Đọc kèm: `CLAUDE.md` (yêu cầu nghiệp vụ) và `ARCHITECT.md` (quy tắc ranh giới).

MỤC LỤC

1. Hệ thống này làm gì?
2. Toàn cảnh trong một sơ đồ
3. Thư mục cấp cao nhất
4. `backend/` — phần lớn nhất
5. `airflow/` — chạy lại theo lịch
6. `frontend/` — giao diện web
7. `eval/` — chấm điểm hệ thống
8. `scripts/check_conventions.py`
9. Hạ tầng
10. Xâu chuỗi lại: một lượt chạy từ đầu đến cuối
11. Đọc tiếp từ đâu

I Hệ thống này làm gì?

Hãy hình dung bạn là một kỹ sư dữ liệu. Sếp đưa bạn 7 file CSV và nói "*dựng cho tôi dashboard xem tỉ lệ sinh viên bỏ học theo khóa*". Bạn sẽ mất vài ngày: mở từng file xem có gì, nghĩ xem nên gom bảng thế nào, viết SQL biến đổi, viết test, chạy thử, tối ưu query chậm, rồi mới dựng được biểu đồ.

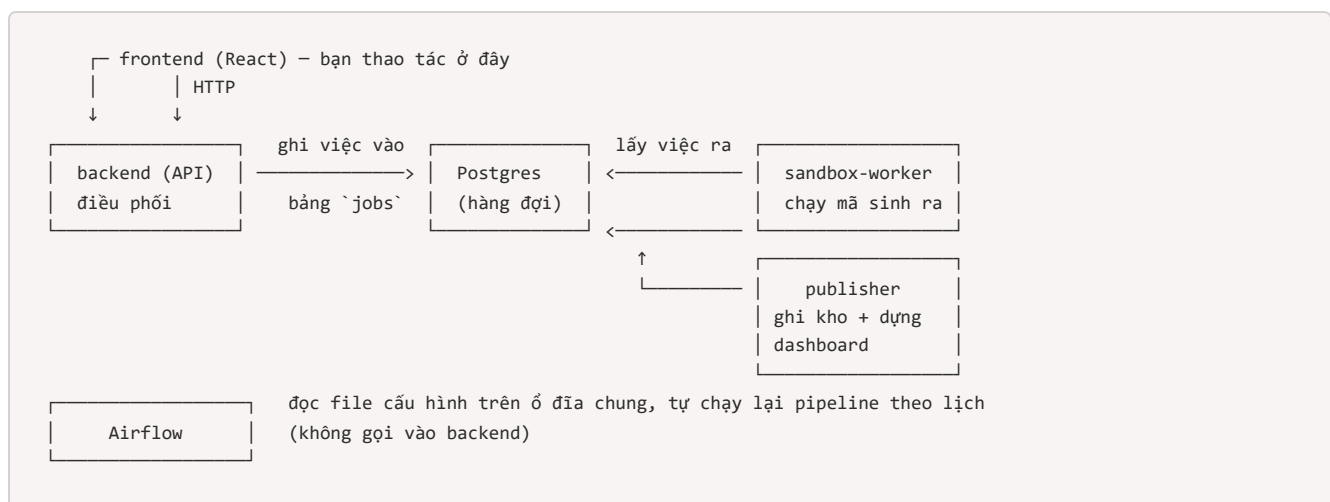
DataForge làm **phần lập đi lập lại** của công việc đó, nhưng giữ lại **phần cần phán đoán** cho con người:

Bạn đưa vào: 7 file CSV + "cho tôi xem tỉ lệ bỏ học theo khóa"

Hệ thống trả: 1. Kho dữ liệu đã dựng sẵn trên Postgres
2. Dashboard Metabase xem được ngay
3. Toàn bộ mã nguồn pipeline (tải ZIP hoặc mở Pull Request)
– chạy độc lập được trên máy bạn, không cần DataForge nữa

Điểm mấu chốt khiến dự án này khác một con bot sinh code thông thường: **mã sinh ra được đem chạy thật rồi đối chiếu kết quả**, và có hai điểm dừng bắt buộc để con người duyệt trước khi bất cứ thứ gì được công bố.

2 Toàn cảnh trong một sơ đồ



Điều quan trọng nhất cần nhớ: các tiến trình này **không gọi hàm của nhau**. Chúng để lại việc cho nhau trong một cái bảng tên là `jobs` trong Postgres – giống như để giấy nhắn trên bàn thay vì gọi điện. Nhờ vậy, tắt bật lại bất kỳ tiến trình nào cũng không mất việc đang dở.

Vì sao phải tách ra nhiều tiến trình như vậy?

Vì **mã do AI sinh ra không được phép chạy chung chỗ với backend**. Backend giữ mật khẩu database, API key. Nếu một câu SQL do AI viết ra chạy ngay trong backend, nó có thể đọc trộm file, ngốn hết RAM, hoặc ghi bậy vào kho dữ liệu thật. Nên mã sinh ra bị đẩy sang `sandbox-worker` —

một tiến trình bị cắt hết quyền: không mạng internet, không mật khẩu kho dữ liệu, bị giới hạn RAM/CPU/thời gian.

3 Thư mục cấp cao nhất

THƯ MỤC / FILE	DÙNG ĐỂ LÀM GÌ
backend/	Toàn bộ Python: API, bộ điều phối, các agent AI, worker, publisher
frontend/	Giao diện web React bạn nhìn thấy và bấm vào
airflow/	Bộ chạy lại pipeline theo lịch (hàng ngày, hàng tuần...)
eval/	Bộ chấm điểm chất lượng hệ thống, chạy riêng, không phải web app
scripts/	Công cụ nội bộ cho lập trình viên
infra/postgres/	Script tạo database và phân quyền lúc khởi tạo Postgres
docker-compose.yml	Công thức dựng cả hệ thống bằng một lệnh
CLAUDE.md	Yêu cầu nghiệp vụ + quy ước viết code
ARCHITECT.md	Luật về ranh giới: ai được gọi ai, ai giữ quyền gì
SECURITY.md	Kênh báo lỗ hổng bảo mật
.pre-commit-config.yaml	Kiểm tra tự động trước mỗi lần commit
.github/workflows/ci.yml	Kiểm tra tự động trên GitHub mỗi lần push
.github/dependabot.yml	Tự động đề xuất nâng cấp thư viện hàng tuần

4 backend/ — phần lớn nhất

4.1. Cấu hình của chính dự án backend

FILE	VAI TRÒ
<code>pyproject.toml</code> thật	Khai báo mọi thư viện kèm phiên bản chính xác (FastAPI 0.141.1, DuckDB 1.5.5...) và cấu hình cho Ruff / mypy / pytest. Ghim cứng phiên bản để hôm nay chạy được thì sang tháng vẫn chạy được y hệt.
<code>uv.lock</code> thật	Khóa chính xác cả cây phụ thuộc, kể cả thư viện con.
<code>.python-version</code> thật	Ghi <code>3.11.14</code> , khớp với base image Docker.
<code>Dockerfile</code> thật	Công thức đóng gói backend thành image; chạy bằng user thường (không phải root), base image ghim cả tag lẫn digest.
<code>alembic.ini</code> thật	Cấu hình công cụ quản lý thay đổi cấu trúc database.

4.2. Luật quan trọng nhất: ai được gọi ai

Tưởng tượng backend có hai loại module:

- Loại "suy đoán" — gọi AI, kết quả có thể khác nhau mỗi lần: `app/agents/`.
- Loại "chắc chắn" — cùng đầu vào luôn cho cùng đầu ra: `profiler`, `compare`, `optimizer/judge`, `codegen`, `publisher`, `metabase`, `worker`.

Loại "chắc chắn" bị cấm gọi loại "suy đoán" (trừ việc dùng chung định nghĩa dữ liệu). Vì sao? Hai lý do:

- Không cho AI tự chấm điểm chính nó.** Nếu `optimizer/judge.py` được phép gọi AI hỏi "bản viết lại này có đúng không?" thì cả hệ thống mất giá trị — AI sẽ gật đầu với chính đề xuất của nó. Phán quyết giữ/loại phải do code chạy thật rồi đo, không do AI nói.
- Worker không có API key.** `sandbox-worker` chạy trong container không có `GEMINI_API_KEY`. Nếu lỡ import nhầm, test trên máy vẫn xanh nhưng chạy thật sẽ sập.

Sơ đồ phụ thuộc chính thức (ở ARCHITECT.md):

```
api → services → graph → agents → (Gemini)
      ↓
      ↳ codegen, profiler, compare, optimizer, jobs
worker → profiler, compare, codegen, publisher, metabase (KHÔNG có agents)
```

4.3. app/main.py — điểm khởi động của API một nửa thật

`create_app()` (thật) dựng ứng dụng FastAPI: bật CORS cho frontend gọi vào, rồi gắn 6 nhóm route (health, auth, projects, batches, runs, exports) dưới đường dẫn /api.

`lifespan()` (chỗ trống) là phần chạy lúc bật và lúc tắt server. Khi hoàn thiện, nó sẽ: mở kết nối Postgres → mở bộ lưu trạng thái của LangGraph → dựng graph → **khôi phục các lượt chạy đang dở** (quan trọng: restart server không làm mất việc) → chạy nền một vòng lặp chuyển kết quả từ worker về cho graph.

4.4. app/core/ — cấu hình và tiện ích chung

FILE	TRẠNG THÁI	NỘI DUNG
config.py	thật	Đây là nơi ranh giới bảo mật được cài đặt thật sự. Có ba lớp cấu hình riêng: ApiSettings (backend — có key Gemini, Qdrant, GitHub), WorkerSettings (sandbox — cố tình không có ô nào để điền mật khẩu kho dữ liệu), PublisherSettings (lớp duy nhất có warehouse_dsn). Worker không thể vô tình chạm vào kho dữ liệu vì cấu hình của nó không tồn tại trường đó. Còn có OptimizerBudget giới hạn vòng tối ưu (tối đa 5 ứng viên, 600 giây...).
security.py	chỗ trống	Băm mật khẩu bằng argon2, phát và kiểm tra vé đăng nhập JWT.
logging.py	chỗ trống	Cấu hình ghi log có cấu trúc cho cả ba tiến trình.

4.5. app/db/ — dữ liệu ứng dụng

`models.py` (thật) — bản thiết kế của 14 bảng trong database dataforge_app. Vài bảng đáng chú ý và vì sao chúng tồn tại:

BẢNG	GIỮ CÁI GÌ	VÌ SAO CẦN
<code>users</code> , <code>projects</code> , <code>project_members</code>	Người dùng, dự án, vai trò	Phân biệt Engineer (làm) và Reviewer (duyệt)
<code>batches</code> , <code>source_files</code>	Lô file CSV đã tải lên	Lưu nguyên vẹn file gốc + cấu hình đọc, để tái tạo được về sau
<code>pipeline_runs</code>	Một lượt chạy pipeline	Cột <code>pending</code> (JSON) giữ nội dung đang chờ bạn duyệt, nên mở lại trình duyệt vẫn thấy đúng chỗ đang dừng
<code>design_versions</code> , <code>code_versions</code>	Từng phiên bản thiết kế và mã	Sửa gì cũng tạo phiên bản mới, không ghi đè
<code>execution_reports</code>	Kết quả chạy thử	Bảng chứng để Reviewer xem ở Cổng 2
<code>rewrite_attempts</code>	Nhật ký mọi lần thử tối ưu	Kể cả lần bị loại , kèm lý do — để giải trình được
<code>approvals</code>	Lịch sử phê duyệt	Cột <code>fingerprint</code> khóa phê duyệt vào đúng một phiên bản cụ thể
<code>jobs</code>	Hàng đợi việc	Trái tim giao tiếp giữa các tiến trình
<code>published_versions</code>	Phiên bản đã công bố	Biết bản nào đang phục vụ dashboard
<code>metabase_objects</code>	ID của card/dashboard trên Metabase	Chạy lại không tạo dashboard trùng

Hai chi tiết kỹ thuật nhỏ nhưng quan trọng:

- `Approval.fingerprint` — "dấu vân tay" gộp từ thiết kế + mã + test + lô dữ liệu + báo cáo. **Sửa một dòng code sau khi duyệt → vân tay đổi → phê duyệt cũ tự động hết hiệu lực.** Đây là cách chặn kịch bản "duyệt bản A rồi lên công bố bản B".
- `jobs` có một index đặc biệt `uq_jobs_running_serial_key` chỉ áp dụng cho job đang chạy. Tác dụng: cùng một pipeline không bao giờ có hai lượt chạy song song — database tự chặn ở tầng thấp nhất, không phụ thuộc code nhớ kiểm tra.

`session.py` (thật) — hai hàm nhỏ tạo kết nối Postgres kiểu bất đồng bộ.

`checkpointer.py` (thật) — chạy riêng bằng `python -m app.db.checkpointer`, tạo các bảng mà LangGraph dùng để lưu trạng thái graph. **Cố tình không đi qua Alembic** vì LangGraph tự quản lý cấu trúc bảng của nó; `migrations/env.py` được dẫn bỏ qua các bảng này để Alembic không tưởng nhầm là bảng thừa rồi xóa mất.

4.6. `app/versioning.py` chỗ trống — cơ chế "dấu vân tay"

Định nghĩa `VersionParts` gồm 6 mảnh: `design_hash`, `code_hash`, `tests_hash`, `batch_id`, `execution_job_id`, `report_hash`. Hàm `fingerprint()` gộp tất cả thành một chuỗi hash.

Khi Reviewer bấm duyệt, hệ thống chụp lại vân tay lúc đó. Trước khi công bố, nó tính lại vân tay hiện tại và so. Lệnh → ném `StaleApprovalError` kèm **danh sách chính xác mảnh nào đã đổi**, và bắt duyệt lại.

4.7. `app/api/` — cửa ngõ HTTP

`schemas.py` (thật) — định nghĩa hình dạng dữ liệu đi vào/đi ra qua HTTP.

⚠ Repo có **ba** file tên `schemas.py`, rất dễ nhầm. Chúng phục vụ ba mục đích khác nhau và **không được import chéo**:

FILE	MÔ TẢ CÁI GÌ	AI DÙNG
<code>api/schemas.py</code>	Dữ liệu qua HTTP	Route + frontend (<code>types.ts</code> phải khớp)
<code>agents/schemas.py</code>	Dữ liệu AI phải trả về	Agent, codegen, graph
<code>profiler/schemas.py</code>	Hồ sơ thống kê của dữ liệu	Profiler → Modeler

Cần chuyển đổi giữa chúng thì viết hàm ở `services/`.

`deps.py` (gần hết là chỗ trống) — các "người gác cổng" dùng chung cho route: lấy kết nối database, giải mã vé đăng nhập, và `require_role(...)` kiểm tra bạn có đúng vai trò trong dự án không. Đây là nơi thực thi luật "*chỉ Reviewer mới được duyệt*".

`routes/` — mỗi file một nhóm endpoint:

FILE	ENDPOINT	TRẠNG THÁI
<code>health.py</code>	GET <code>/health</code>	thật — Docker dùng để biết backend đã sẵn sàng chưa
<code>auth.py</code>	Đăng ký, đăng nhập	chỗ trống
<code>projects.py</code>	Tạo dự án, thêm thành viên, xem lịch sử duyệt	chỗ trống
<code>batches.py</code>	Tạo lô, tải CSV lên, xác nhận cách đọc file, đánh dấu lô sẵn sàng	chỗ trống
<code>runs.py</code>	Nhóm lớn nhất, 9 endpoint — toàn bộ vòng đời một lượt chạy	chỗ trống
<code>exports.py</code>	Tải ZIP, mở Pull Request	chỗ trống

Chi tiết `runs.py` vì nó phản ánh đúng luồng bạn thao tác:

```

POST /runs          bắt đầu chạy
GET /runs/{id}      hỏi "xong chưa?" — frontend gọi lại mỗi 2 giây
POST /runs/{id}/clarifications trả lời câu hỏi AI thắc mắc
POST /runs/{id}/design-decision — Cổng 1: duyệt / sửa / trả lại thiết kế
GET /runs/{id}/code  xem mã sinh ra + so sánh với bản trước
PUT /runs/{id}/code  tự tay sửa mã (tạo phiên bản mới, hủy phê duyệt cũ)
GET /runs/{id}/evidence xem bằng chứng: kết quả test, bảng so hiệu năng
POST /runs/{id}/code-decision — Cổng 2: Reviewer duyệt
POST /runs/{id}/dashboard/retry chạy lại riêng bước dashboard nếu nó lỗi

```

Route chỉ nhận request rồi gọi xuống `services/` — không chứa logic nghiệp vụ.

4.8. `app/graph/` — bộ não điều phối

Đây là phần quan trọng nhất nếu bạn muốn hiểu hệ thống chạy thế nào.

`state.py` (thật) — `PipelineState` là "tờ giấy" đi theo suốt lượt chạy, mỗi bước ghi thêm vào: hồ sơ dữ liệu → thiết kế → đã duyệt chưa → mã phiên bản mấy → báo cáo chạy → nhật ký tối ưu → link dashboard. LangGraph tự lưu tờ giấy này xuống Postgres sau **mỗi** bước, nên sập server giữa chừng vẫn chạy tiếp được đúng chỗ.

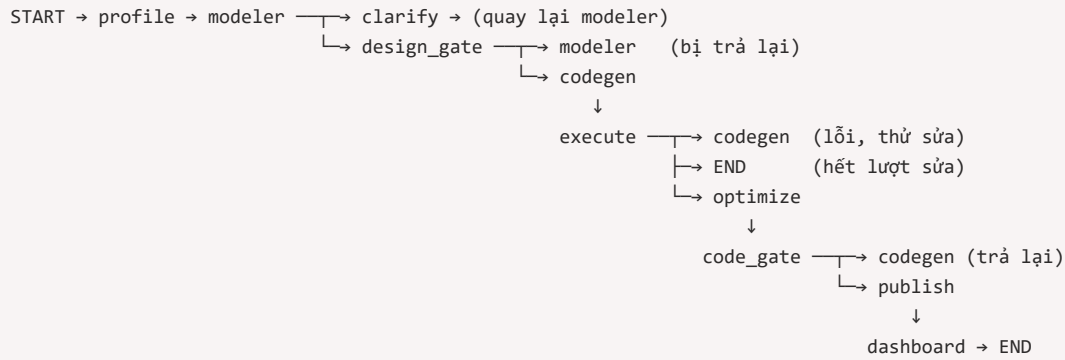
`builder.py` (thật — đã nối dây đủ) — đăng ký 10 bước và vẽ đường đi giữa chúng. Khi cần biết "sau bước X thì đi đâu", đọc file này đầu tiên.

`nodes.py` (toàn bộ chỗ trống, nhưng docstring là bản hợp đồng chi tiết) — 10 bước:

BƯỚC	LÀM GÌ	CÓ DỪNG CHỜ NGƯỜI KHÔNG?
<code>profile_node</code>	Nhờ worker khảo sát dữ liệu	Chờ worker
<code>modeler_node</code>	Nhờ AI đề xuất thiết kế	—
<code>clarify_node</code>	Hỏi lại bạn khi AI không chắc	Dừng chờ bạn
<code>design_gate_node</code>	Cổng 1	Dừng chờ duyệt
<code>codegen_node</code>	Nhờ AI viết mã (chỉ khi đã duyệt)	—
<code>execute_node</code>	Nhờ worker chạy thử mã	Chờ worker
<code>optimize_node</code>	Thử viết lại SQL cho nhanh hơn	Chờ worker
<code>code_gate_node</code>	Cổng 2	Dừng chờ Reviewer
<code>publish_node</code>	Nhờ publisher nạp vào Postgres	Chờ publisher
<code>dashboard_node</code>	Nhờ publisher dựng Metabase	Chờ publisher

Cộng thêm 4 hàm `route_after_*` quyết định rẽ nhánh. Ví dụ `route_after_execute`: chạy thử **thất bại** → quay lại `codegen` để AI tự sửa, nhưng chỉ trong giới hạn số lần cho phép, hết lượt thì dừng hẳn và báo lỗi.

Luồng đầy đủ:



4.9. app/agents/ — phân duy nhất gọi AI

Ranh giới cứng: các module ở đây chỉ nhận `DataProfile` — tức bảng thống kê (cột này có bao nhiêu % rỗng, bao nhiêu giá trị khác nhau...). Chúng không bao giờ nhận đường dẫn file hay dòng dữ liệu thật. Dữ liệu của bạn không bị gửi cho Google.

FILE	TRẠNG THÁI	NỘI DUNG
<code>llm.py</code>	chờ trống	Lớp bọc gọi Gemini: ép AI trả JSON đúng khuôn, sai thì báo lỗi và bắt làm lại (tối đa 2 lần), đếm token đã dùng
<code>schemas.py</code>	thật	Khuôn mà AI bắt buộc phải điền — xem bên dưới
<code>modeler.py</code>	chờ trống	Đề xuất thiết kế dữ liệu
<code>codegen.py</code>	chờ trống	Viết mã dbt — chỉ tầng silver và mart
<code>optimizer.py</code>	chờ trống	Đề xuất viết lại SQL cho nhanh hơn
<code>prompts/modeler.md</code>	thật	Lời dặn cho Modeler
<code>prompts/codegen.md</code>	thật	Lời dặn cho Codegen
<code>prompts/optimizer.md</code>	thật	Lời dặn cho Optimizer

`schemas.py` **đáng đọc kỹ** vì nó là cách "ép AI phải suy nghĩ đủ". `DataDesign` bắt buộc có: bảng fact (kèm **grain** — một dòng nghĩa là gì), bảng dimension, khóa, quan hệ, định nghĩa chỉ số, quy tắc làm sạch, chính sách cập nhật, yêu cầu dashboard, và `constraints`. **Mỗi mục đều bắt buộc có trường `rationale`** — AI phải giải thích vì sao chọn vậy, không được phán bừa.

`ModelerOutput` có hai ô: `design` hoặc `questions` — không bao giờ cả hai. Nghĩa là khi dữ liệu mơ hồ (cột `amount` là đơn giá, thành tiền, hay số đã trả?), AI **phải hỏi lại** thay vì đoán.

Xem thử `prompts/modeler.md` để thấy tinh thần đó được viết thẳng vào lời dặn:

"Columns with `has_leading_zeros` are codes, not numbers." — cột có số 0 ở đầu (`00123`) là mã, không phải số.

4.IO. `app/profiler/` — khảo sát dữ liệu (chắc chắn, không AI)

`schemas.py` (thật) — `DataProfile`, với docstring ghi thẳng: *"The only data description ever sent to the LLM. Never add raw rows here."*

Gồm:

- `ColumnProfile`: tên cột, kiểu suy ra, % rỗng, số giá trị khác nhau, min/max, tối đa 5 giá trị mẫu, và cờ `has_leading_zeros`.
- `KeyCandidate`: cột nào có thể làm khóa chính (kể cả khóa ghép 2 cột).
- `RelationshipCandidate`: hai bảng có thể nối với nhau không — kèm **bằng chứng**: tỉ lệ khớp, số bản ghi mờ côi, và `fanout_warning` cảnh báo nối xong số dòng bị phình lên.

`profile.py` (chỗ trống) — dùng DuckDB quét file CSV và điền ra `DataProfile`. Chạy **trong sandbox worker**, không chạy trong API.

4.II. `app/compare/` — hai phép kiểm chứng *tách biệt*

Đây là ý tưởng cốt lõi của dự án, đáng dừng lại một chút.

Khi tối ưu một câu SQL, có **hai câu hỏi hoàn toàn khác nhau**:

1. Kết quả có đúng không? → so với đáp án chuẩn.
2. Bản viết lại có ra kết quả giống bản gốc không? → so bản mới với bản cũ.

Chúng **không được gộp**. Lý do rất tinh tế: nếu bản gốc vốn đã sai, mà bản viết lại cũng sai y hệt, thì phép so "mới vs cũ" sẽ báo *"giống nhau, đạt!"* — nhưng pipeline vẫn sai. Giữ nguyên một kết quả sai không làm nó đúng.

FILE	TRẠNG THÁI	KIỂM CHỨNG GÌ
<code>multiset.py</code>	chỗ trống	TƯƠNG ĐƯƠNG: schema khớp, rồi so dữ liệu bằng <code>EXCEPT ALL</code> hai chiều — đếm cả số lần lặp của từng dòng, không chỉ đếm tổng số dòng. Có quy tắc riêng cho NULL, số thực (làm tròn), timestamp.
<code>benchmark.py</code>	chỗ trống	NHANH HƠN: chạy lặp nhiều lần lấy trung vị. Chỉ kết luận nhanh hơn khi mức cải thiện vượt cả ngưỡng tối thiểu lần nhiều đo được.

Vì sao phải so multiset chứ không đếm dòng? Vì hai bảng cùng có 1000 dòng vẫn có thể khác nhau hoàn toàn về nội dung.

Vì sao phải tính nhiều? Vì máy tính chạy nhanh chậm thất thường. Nếu không trừ nhiều, một bản viết lại *không cải thiện gì* vẫn có thể tình cờ đo được "nhanh hơn 3%" rồi được giữ nhầm.

4.12. `app/optimizer/` — quan tòa (chắc chắn, tuyệt đối không AI)

`judge.py` (chỗ trống) — nhận kết quả đo từ worker và ra phán quyết, xét theo đúng thứ tự: lỗi khi chạy → quá giờ → sai kết quả → không nhanh hơn → **giữ**. Docstring ghi rõ: **"Never call an LLM here."**

`select_best()` chọn bản nhanh nhất trong số những bản hợp lệ. **Nếu mọi ứng viên đều bị loại thì giữ nguyên bản gốc** — "query vốn đã tốt, không cần sửa" là một kết quả hợp lệ, không phải thất bại.

`rules.py` (chỗ trống) — dùng thư viện sqlglot sinh ứng viên viết lại theo quy tắc cố định, không cần AI. Điểm hay: các ứng viên này **đi qua đúng cái quan tòa đó** như ứng viên của AI. Không có đường tắt nào cho ai cả.

4.13. `app/codegen/` — sinh mã theo khuôn (chắc chắn, không AI)

Đừng nhầm với `app/agents/codegen.py` (AI viết mã). Thư mục này dựng phần **có khuôn mẫu cố định**, nơi dùng AI chỉ tổ rước rủi ro mà chẳng được gì.

FILE	TRẠNG THÁI	NỘI DUNG
<code>bronze.py</code>	chờ trống	Sinh tầng bronze: đọc CSV với mọi cột để dạng chữ , giữ nguyên <code>00123</code> không biến thành <code>123</code> . Gắn thêm 4 cột truy vết: lô nào, file nào, dòng số mấy, nạp lúc nào.
<code>tests.py</code>	chờ trống	Dịch chỉ những ràng buộc đã được duyệt ở Cổng 1 thành file test của dbt. Quan sát thống kê không tự động thành test bắt buộc.
<code>guard.py</code>	chờ trống	Kiểm tra mã AI sinh ra trước khi đem chạy
<code>project.py</code>	chờ trống	Gộp mọi thứ thành một dự án dbt hoàn chỉnh

Vì sao `tests.py` **chỉ dùng ràng buộc đã duyệt**? Đây là một điểm tinh tế. Profiler có thể thấy "cột *email* không có giá trị trùng nào". Nhưng đó là quan sát **trên đúng lô dữ liệu này**, không phải luật. Lô sau có thể có email trùng hợp lệ. Nếu tự động biến quan sát thành test, pipeline sẽ gãy oan. Nên: quan sát chỉ được dùng để **đề xuất** cho con người, và chỉ khi con người gật ở Cổng 1 thì mới thành test.

`guard.py` chặn theo kiểu *danh sách cho phép* (không phải danh sách cấm): chỉ chấp nhận file trong `models/silver/` và `models/mart/`; chỉ cho gọi `ref`, `config`, `var`; SQL chỉ được là `select` / `with`; cấm hàm đọc file, cấm hàm ngẫu nhiên như `now()`, `random()` (vì chúng làm kết quả mỗi lần chạy một khác, không so sánh được). Docstring nói rõ đây chỉ là **lớp phòng thủ thêm** — hàng rào thật là sandbox.

Vì sao dùng danh sách cho phép? Vì danh sách cấm luôn thiếu — bạn không thể liệt kê hết mọi thứ nguy hiểm, nhưng liệt kê được vài thứ an toàn.

`project.py` gộp: bronze (khuôn) + test (từ ràng buộc đã duyệt) + mã AI (đã qua guard) + template → một phiên bản dbt hoàn chỉnh, và tính `code_hash` với `tests_hash` **riêng biệt** để biết chính xác cái gì đã đổi. Nó cũng sinh `dag_config.json` cho Airflow.

4.14. `app/ingest/` — nhận file CSV

`sniff.py` (chờ trống) — đoán cách đọc file: mã hóa gì (UTF-8? UTF-16?), phân cách bằng dấu phẩy hay chấm phẩy, đâu là ký hiệu rỗng (`NULL`, `NA`, `?`...), định dạng ngày ra sao. Rồi trả về vài dòng xem trước để bạn xác nhận — vì đoán tự động có thể sai.

storage.py (bố cục **thật** trong docstring, thao tác file là chỗ trống) — quy định **một** cấu trúc thư mục duy nhất trên ổ đĩa dùng chung:

projects/<dự án>/batches/<lô>/raw/<file.csv>	file gốc, không sửa gì
projects/<dự án>/batches/<lô>/_READY	cờ báo "lô này đủ file rồi"
projects/<dự án>/runs/<lượt>/code/v<N>/	mã từng phiên bản
projects/<dự án>/runs/<lượt>/workspaces/<job>/	chỗ làm việc tạm của worker
projects/<dự án>/approved/<phiên bản>/	mã đã duyệt — Airflow đọc ở đây

File **_READY** nhỏ mà quan trọng: nó là cách giải bài toán "một lô cần nhiều file, chạy khi nào?". Airflow chờ đúng cờ này, không đoán mò.

4.15. **app/publisher/** và **app/metabase/** — chỉ publisher chạm tới

publisher/postgres.py (chỗ trống) — nạp dữ liệu vào Postgres theo ba nhịp, và thứ tự này là có lý do:

1. load_staging chép vào một schema tạm (chưa ai nhìn thấy)
2. reconcile đếm dòng + so checksum giữa nguồn và đích
3. swap_serving_views đổi "công tắc" trở sang schema mới — trong MỘT transaction

Nếu bước 2 phát hiện lệch, dừng luôn, **không bước sang 3**. Nếu bước 3 hỏng giữa chừng, transaction quay lui và **dashboard vẫn phục vụ bản cũ bình thường** — người dùng cuối không thấy màn hình trắng.

metabase/client.py (chỗ trống) — gọi API Metabase: đăng nhập, đồng bộ schema, tạo/sửa card, chạy thử card, tạo dashboard.

metabase/builder.py (chỗ trống) — điều phối: đồng bộ → tạo card → **chạy thử từng card xem có ra số không** → gom vào dashboard → trả link.

Mẹo quan trọng: trước khi tạo card, nó tra bảng **metabase_objects** theo khóa logic. Đã có thì **sửa**, chưa có thì **tạo mới**. Nhờ vậy bấm thử lại 5 lần cũng chỉ có một dashboard, không phải năm cái trùng nhau.

4.16. `app/export/` — mang mã nguồn về nhà

`bundle.py` (chỗ trống) — dựng gói chạy độc lập: dự án dbt, bộ nạp CSV, script đẩy sang Postgres, cấu hình dashboard kèm script dựng lại, Dockerfile + Compose ghim phiên bản, file DAG, manifest phiên bản và báo cáo kiểm chứng.

Tiêu chuẩn nghiệm thu rất rõ: **mang ZIP sang máy sạch, điền cấu hình, đưa CSV vào → dựng lại được mart và dashboard mà không gọi một API nào của DataForge**. Gói xuất không được phụ thuộc ngược vào nền tảng.

`build_zip()` tạo ZIP **tất định** — sắp xếp file theo thứ tự cố định, đặt timestamp cố định. Nên cùng một phiên bản nén lại bao nhiêu lần cũng ra file giống hệt, checksum không đổi.

`github_pr.py` (chỗ trống) — tạo nhánh, commit, mở Pull Request. **Không bao giờ tự merge** — quyền đó thuộc về bạn.

4.17. `app/retrieval/store.py` chỗ trống — cho AI xem bài mẫu

Trước khi bảo AI viết mã, hệ thống tìm vài ví dụ tương tự trong kho Qdrant để đưa kèm vào ngữ cảnh — giống như cho xem bài mẫu trước khi làm bài.

Điểm cần chú ý: `search()` có tham số `exclude_datasets`. Khi chấm điểm hệ thống, phải loại đúng bộ dữ liệu đang chấm ra khỏi kho bài mẫu — **không thì AI chỉ đang chép lại đáp án và điểm số trở nên vô nghĩa**.

4.18. `app/jobs/queue.py` chỗ trống — hàng đợi bằng bảng Postgres

Dự án **cố ý không dùng Redis/Celery**. Lý do: đã có Postgres rồi, thêm một hệ thống nữa là thêm một thứ để vận hành, để hỏng, để học.

HÀM	VIỆC
<code>enqueue</code>	Thêm việc; có <code>idempotency_key</code> nên gọi hai lần không tạo hai việc
<code>claim_next</code>	Nhận việc bằng <code>SELECT ... FOR UPDATE SKIP LOCKED</code> — nhiều worker cùng lấy mà không giẫm chân nhau
<code>heartbeat</code>	Báo "tôi còn sống, đang làm"
<code>complete</code> / <code>fail</code>	Xong / hỏng (hỏng thì thử lại có giãn cách)
<code>reclaim_expired</code>	Thu hồi việc của worker chết giữa chừng

`SKIP LOCKED` là mấu chốt: nó bảo Postgres "dòng nào đang bị khóa thì bỏ qua, đưa tôi dòng tiếp theo". Không có nó, các worker sẽ xếp hàng chờ nhau.

`heartbeat` + `reclaim_expired` giải bài toán worker bị kill đột ngột: không ai báo cáo gì cả, nhưng hết hạn thuê thì việc tự quay lại hàng đợi cho worker khác nhặt.

4.19. **backend/worker/** — nơi mã sinh ra thật sự chạy

FILE	TRẠNG THÁI	NỘI DUNG
<code>main.py</code>	chỗ trống	Vòng lặp của sandbox worker: thu hồi việc quá hạn → nhận việc → báo sống → giao cho <code>runner.py</code> → báo kết quả
<code>publisher_main.py</code>	chỗ trống	Vòng lặp của publisher: chỉ nhận <code>publish</code> và <code>metabase_sync</code> ; luôn kiểm tra vân tay phê duyệt trước khi làm
<code>runner.py</code>	chỗ trống	Phân việc theo loại: <code>profile</code> , <code>dbt_build</code> , <code>rewrite_benchmark</code>
<code>sandbox.py</code>	chỗ trống hàng số thật	Lồng cách ly — xem bên dưới

`sandbox.py` là hàng rào bảo mật thật sự của cả hệ thống. Hai hàng số đã viết sẵn:

- `ENV_ALLOWLIST` — tiến trình con **chỉ** được nhận 7 biến môi trường hệ thống cơ bản. Docstring nhấn mạnh: "Never inherit `os.environ`: it holds the app database URL and API keys." Không phải lọc bỏ thứ nguy hiểm, mà là **bắt đầu từ trống rỗng** rồi thêm vào thứ an toàn.

- `DUCKDB_LOCKDOWN_SETTINGS` — tắt truy cập file/mạng của DuckDB, cấm tự cài extension, rồi **khóa luôn cấu hình lại** để chính mã sinh ra không mở lại được.

Cộng thêm: giới hạn RAM/CPU/dung lượng/số tiến trình con, hết giờ thì giết cả cây tiến trình, và **không bao giờ dùng** `shell=True`.

Một điểm thiết kế đáng chú ý ở `runner.py`: `run_rewrite_benchmark` **đo nhưng không phán xét**. Nó chạy cả SQL gốc lẫn SQL ứng viên trên cùng một file DuckDB, gom số liệu thô, rồi gửi về. Việc kết luận giữ hay loại do `optimizer/judge.py` ở phía API làm. Tách đo khỏi xử giúp phán quyết luôn nhất quán và kiểm tra lại được.

4.20. `backend/templates/dbt_project/` — khuôn dbt dùng chung

FILE	NỘI DUNG
<code>dbt_project.yml.j2</code>	Khai báo 3 tầng <code>bronze</code> / <code>silver</code> / <code>mart</code> , các biến <code>raw_dir</code> , <code>batch_id</code> , <code>loaded_at</code>
<code>profiles.yml.j2</code>	Cấu hình kết nối DuckDB qua biến môi trường
<code>macros/dataforge_tests.sql</code>	Hai test tự viết: <code>dataforge_unique_combination</code> (nhiều cột gộp lại phải duy nhất) và <code>dataforge_expression_is_true</code> (một điều kiện SQL phải luôn đúng)
<code>macros/generate_schema_name.sql</code>	Bắt dbt đặt tên schema đúng như khai báo, không thêm tiền tố mặc định

Để ý `loaded_at` là **biến truyền vào**, không phải `now()`. Vì `now()` khiến mỗi lần chạy ra kết quả khác nhau, không thể so sánh hai lần chạy với nhau được.

4.21. `backend/migrations/` — quản lý thay đổi cấu trúc database

`env.py` (**thật**) cấu hình Alembic, và quan trọng là dặn nó **bỏ qua các bảng** `checkpoint*` (của LangGraph). `versions/20260929_..._baseline.py` là migration đầu tiên tạo toàn bộ 14 bảng.

4.22. `backend/tests/`

`conftest.py` (**thật**) — đồ nghề dùng chung cho test. `unit/` chạy không cần dịch vụ ngoài. `integration/` cần Postgres thật và **tự bỏ qua** nếu chưa cấu hình.

Dự án **cấm dùng SQLite thay Postgres trong test**, dù SQLite tiện hơn nhiều. Vì SQLite không có `SKIP LOCKED`, không có JSONB, không có enum – mà đó đúng là những thứ hệ thống dựa vào. Test xanh trên SQLite sẽ là lời hứa giả.

5 airflow/ — chạy lại theo lịch

FILE	TRẠNG THÁI	NỘI DUNG
<code>Dockerfile</code>	thật	Airflow 3.3.2, và dbt cài trong virtualenv riêng <code>/opt/dbt-venv</code>
<code>dag/dataforge_pipelines.py</code>	khung thật logic trống	Một file DAG khuôn mẫu duy nhất
<code>dbt/requirements.in</code> / <code>.txt</code>	thật	Thư viện dbt ghim phiên bản, có hash kiểm tra
<code>tests/test_dag_integrity.py</code>	chỗ trống	Kiểm tra file DAG import được
<code>ruff.toml</code>	thật	Quy tắc lint, thêm bộ rule riêng cho Airflow

Ý tưởng hay nhất ở đây nằm ngay dòng docstring đầu file DAG:

"Fixed DAG template. Generated pipelines only contribute `dag_config.json`, never Python."

Nghĩa là: pipeline mới **không bao giờ sinh ra file Python mới** cho Airflow. Cuối file có một vòng lặp quét `projects/*/approved/*/dag_config.json` và dựng một DAG cho mỗi file cấu hình. Thêm pipeline = ghi thêm một file JSON, **không cần deploy lại Airflow**, và quan trọng hơn: **Airflow không bao giờ phải chạy mã Python do AI viết**.

Các bước trong DAG: chờ lô đủ file (`_READY`) → **kiểm tra schema có đổi không** (đổi thì dừng và hỏi người, không tự áp dụng) → chạy dbt → nhờ publisher nạp dữ liệu.

Hai ràng buộc đáng nhớ:

- **Airflow không giữ mật khẩu kho dữ liệu**. Nó chỉ nhờ publisher làm.
- Mỗi lượt chạy dùng **một file DuckDB riêng**, vì DuckDB chỉ cho một tiến trình ghi tại một thời điểm.

6 frontend/ — giao diện web

FILE	TRẠNG THÁI	NỘI DUNG
<code>package.json</code>	thật	React 19, Vite, TypeScript, Monaco, Biome — ghim phiên bản chính xác
<code>vite.config.ts</code> , <code>tsconfig.json</code> , <code>biome.json</code>	thật	Cấu hình build, kiểu, lint/format
<code>vercel.json</code>	thật	Mọi đường dẫn trả về <code>index.html</code> (kiểu SPA)
<code>index.html</code> , <code>src/main.tsx</code>	thật	Điểm khởi động
<code>src/App.tsx</code>	thật	Khung trang + thanh điều hướng
<code>src/styles.css</code>	thật	CSS thuần, không framework
<code>src/api/types.ts</code>	thật	Kiểu TypeScript khớp 1-1 với <code>api/schemas.py</code>
<code>src/api/client.ts</code>	chỗ trống	16 hàm, mỗi hàm một endpoint backend
<code>src/hooks/useRunPolling.ts</code>	chỗ trống	Hỏi backend "xong chưa" mỗi 2 giây
<code>src/pages/*.tsx</code>	chỗ trống	7 trang

Bảy trang ứng đúng với bảy bước bạn đi qua:

LoginPage	Đăng nhập
ProjectsPage	Chọn hoặc tạo dự án
UploadPage	Tải CSV lên, xác nhận cách đọc file
RunPage	Theo dõi tiến độ, trả lời câu hỏi AI thắc mắc
DesignGatePage	Cổng 1 — xem thiết kế, duyệt / sửa / trả lại
CodeGatePage	Cổng 2 — xem diff mã và bằng chứng (dùng Monaco, tải riêng vì nặng)
ExportPage	Tải ZIP hoặc mở Pull Request

Hai điều đáng chú ý:

- `types.ts` phải khớp `api/schemas.py`. Sửa DTO ở backend mà quên sửa đây thì TypeScript sẽ không cảnh báo — nó không tự biết backend đổi gì.
- Dùng polling, không dùng WebSocket. Vì trạng thái đã lưu bên trong Postgres, hỏi lại luôn ra đúng kết quả; đổi lại không phải nuôi kết nối lâu dài và xử lý đủ thứ tình huống rớt mạng.

7 eval/ — chấm điểm hệ thống

Chạy riêng, không phải một phần của web app.

FILE	TRẠNG THÁI	NỘI DUNG
tasks.py	chỗ trống	Nạp danh sách bài kiểm tra
metrics.py	chỗ trống model thật	TaskScore gom mọi chỉ số
fault_injection.py	chỗ trống danh sách thật	Cố tình làm hỏng dữ liệu
run_eval.py	chỗ trống	Chạy toàn bộ bài kiểm tra
golden/	trống	Chỗ để bộ dữ liệu chuẩn (chưa có)

`fault_injection.py` là ý tưởng đáng chú ý. Test chạy qua hết trên dữ liệu sạch **không chứng minh** bộ test tốt — một bộ test rỗng cũng "qua hết". Nên hệ thống cố tình tiêm 6 loại lỗi (khóa rỗng, khóa trùng, bản ghi mờ côi, trạng thái lạ, ngày sai, số tiền âm) rồi đo xem bộ test **bắt được bao nhiêu**. Đó mới là thước đo thật.

`metrics.py` tái dùng `app.compare.multiset` thay vì viết lại logic so sánh — để bộ chấm điểm và hệ thống thật dùng chung một định nghĩa "giống nhau".

`run_eval.py` có `run_ablation()`: chạy đi chạy lại với từng tính năng bật/tắt (có/không bài mẫu, có/không tối ưu, có/không sinh test) để biết **từng phần đóng góp bao nhiêu**, thay vì chỉ biết tổng thể chạy được.

8 scripts/check_conventions.py

Đây là file **duy nhất trong repo có logic đầy đủ**, vì nó là công cụ chứ không phải nghiệp vụ. Nó bắt hai luật:

1. **Cấm mọi comment trong code** — Python kiểm bằng bộ phân tích cú pháp, các loại file khác bằng regex (có ẩn chuỗi ký tự trước để không báo nhầm dấu `#` nằm trong chuỗi). Ngoại lệ duy nhất: dòng `#!/bin/bash`.
2. **Code chỉ được chứa ký tự ASCII**.

VÌ SAO CẤM COMMENT?

Đây là lựa chọn có chủ đích của dự án: comment hay bị bỏ quên khi code đổi, rồi thành thông tin sai lệch. Thay vào đó, bắt code tự giải thích qua tên hàm và cấu trúc; điều gì buộc phải nói thì viết vào docstring. Tài liệu (`.md`) thì viết thoải mái, kể cả tiếng Việt — như chính file bạn đang đọc.

Chạy trong CI và trong pre-commit.

9 Hạ tầng

9.1. `infra/postgres/init.sh` thật

Chạy một lần khi Postgres khởi tạo. Tạo 5 tài khoản và 4 database **trên cùng một instance** (tách vai trò mà không phải dựng thêm container):

DATABASE	CHỦ	AI VÀO ĐƯỢC
<code>dataforge_app</code>	<code>dataforge</code>	backend, migrate
<code>airflow</code>	<code>airflow</code>	Airflow tự lo
<code>metabase</code>	<code>metabase</code>	Metabase tự lo
<code>dataforge_warehouse</code>	<code>publisher</code>	<code>publisher</code> (ghi), <code>metabase_reader</code> (chỉ đọc)

Phần hay nhất nằm ở mấy dòng cuối: `metabase_reader` bị thu hồi sạch quyền, rồi chỉ được cấp lại quyền kết nối và `SELECT`. Nghĩa là **kể cả có ai đó chiếm được tài khoản Metabase, họ vẫn không ghi được vào kho dữ liệu** — vì Postgres không cho, chứ không phải vì code nhớ kiểm tra.

9.2. `docker-compose.yml` thật

11 dịch vụ. Những điểm đáng chú ý:

- `migrate` chạy **một lần rồi tắt**: cập nhật cấu trúc database rồi tạo bảng checkpoint. Các dịch vụ khác đều chờ nó xong mới khởi động — nên không bao giờ có chuyện backend nhận request khi database chưa sẵn sàng.
- `backend` là nơi duy nhất có key Gemini, Qdrant, GitHub.
- `sandbox-worker` — đây là chỗ mọi luật bảo mật được thực thi ở tầng container: nằm trên mạng `internal` (**không có internet**), hệ thống file **chỉ đọc**, **bỏ toàn bộ capability** của Linux, giới hạn RAM/CPU/số tiến trình, cấm leo thang đặc quyền.
- `publisher` là nơi duy nhất có mật khẩu kho dữ liệu; ổ đĩa gắn **chỉ đọc** (nó đọc file DuckDB worker tạo ra, không tự sinh).
- **Ba container Airflow** riêng (`api-server` , `scheduler` , `dag-processor`) theo kiến trúc Airflow 3.
- Mạng `sandbox` chỉ có Postgres và `sandbox-worker`. Worker **không nhìn thấy** Qdrant, Metabase, Airflow, hay internet.

Đề ý: cùng một luật được cài đặt ở **ba tầng** — cấu hình Python (`WorkerSettings` không có trường), container (mạng internal), và database (phân quyền SQL). Một tầng thùng thì còn hai tầng.

9.3. `.github/workflows/ci.yml` thật

JOB	KIỂM TRA
<code>conventions</code>	Không comment, chỉ ASCII
<code>backend-lint</code>	Format, lint, kiểm kiểu nghiêm ngặt
<code>backend-test</code>	Dựng Postgres thật → migrate → <code>alembic check</code> → tạo checkpoint → <code>alembic check</code> lần nữa → chạy test
<code>frontend</code>	Lint, kiểm kiểu, build thử
<code>images</code>	Build image, thử <code>dbt --version</code> , thử import file DAG
<code>security</code>	Quét lộ secret, quét lỗ hổng thư viện

Chi tiết tinh tế: `alembic check` chạy **hai lần**, trước và sau khi tạo bảng checkpoint. Lần thứ hai để chắc chắn bảng của LangGraph không làm Alembic tưởng nhầm là có thay đổi chưa migrate.

IO Xâu chuỗi lại: một lượt chạy từ đầu đến cuối

1. Bạn tải 7 file CSV → batches.py lưu file nguyên vẹn + đoán cách đọc
2. Bạn xác nhận preview → source_files.read_config_confirmed = true
3. Bạn mô tả yêu cầu, bấm chạy → runs.py tạo PipelineRun, khởi động graph
4. profile_node → bỏ việc vào bảng jobs
→ sandbox-worker nhật, quét bằng DuckDB, trả DataProfile
5. modeler_node → gửi DataProfile (không có dữ liệu thật) cho Gemini
→ nhận về bản thiết kế, hoặc câu hỏi nếu AI chưa chắc
6. clarify_node → [DỪNG] bạn trả lời "amount là thành tiền"
7. design_gate → [DỪNG] == CỔNG 1 == bạn xem, sửa, duyệt
→ ghi Approval + vân tay
8. codegen_node → lấy vài bài mẫu từ Qdrant → Gemini viết silver/mart
→ guard.py kiểm tra → gộp với bronze + test → CodeVersion v1
9. execute_node → sandbox-worker chạy dbt thật, đo thời gian, chạy test
→ lỗi? quay lại bước 8 sửa (trong giới hạn số lần)
10. optimize_node → xếp hạng model chậm → sqlglot + Gemini đề xuất viết lại
→ worker chạy đo → judge.py phán quyết
→ sai kết quả / không nhanh hơn? LOẠI, ghi rõ lý do
11. code_gate → [DỪNG] == CỔNG 2 == Reviewer xem diff + bằng chứng
12. publish_node → publisher: nạp staging → đối chiếu → đối công tắc
13. dashboard_node → publisher: tạo card → chạy thử từng card → dashboard
→ trả về link
14. Bạn tải ZIP / mở PR → toàn bộ pipeline chạy độc lập được trên máy bạn
15. Airflow đọc dag_config.json → tự chạy lại hằng ngày với lô dữ liệu mới

Bốn tình huống bắt buộc phải đúng (tiêu chí nghiệm thu):

TÌNH HUỐNG	HỆ THỐNG PHẢI LÀM GÌ
Bản viết lại nhanh hơn nhưng đổi kết quả	Loại bỏ, giữ bản cũ
Mã bị sửa sau khi đã duyệt	Phê duyệt cũ hết hiệu lực, bắt duyệt lại
Cùng một lô chạy hai lần	Thay thế dữ liệu, không nhân đôi
Tải ZIP sang máy sạch	Dựng lại được mart + dashboard, không gọi API nền tảng

II Đọc tiếp từ đâu

BẠN MUỐN HIỂU

ĐỌC THEO THỨ TỰ

Luồng chạy tổng thể	<code>graph/builder.py</code> (đường đi) → <code>graph/nodes.py</code> (từng bước)
Dữ liệu được lưu thế nào	<code>db/models.py</code>
AI được yêu cầu trả về gì	<code>agents/schemas.py</code> → <code>agents/prompts/*.md</code>
Dữ liệu nào bị gửi cho AI	<code>profiler/schemas.py</code> (chỉ đúng file này)
Vì sao một bản tối ưu bị loại	<code>optimizer/judge.py</code> → <code>compare/multiset.py</code> + <code>compare/benchmark.py</code>
Mã sinh ra bị cách ly ra sao	<code>worker/sandbox.py</code> → phần <code>sandbox-worker</code> trong <code>docker-compose.yml</code>
Ai được phép làm gì	<code>ARCHITECT.md</code> mục 1 — đọc trước khi sửa <code>core/config.py</code> , <code>worker/*</code> , hay Compose
